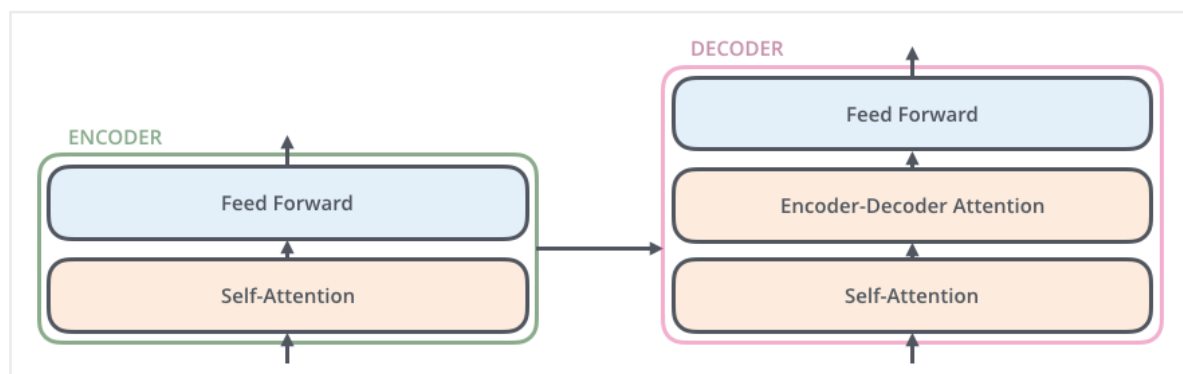
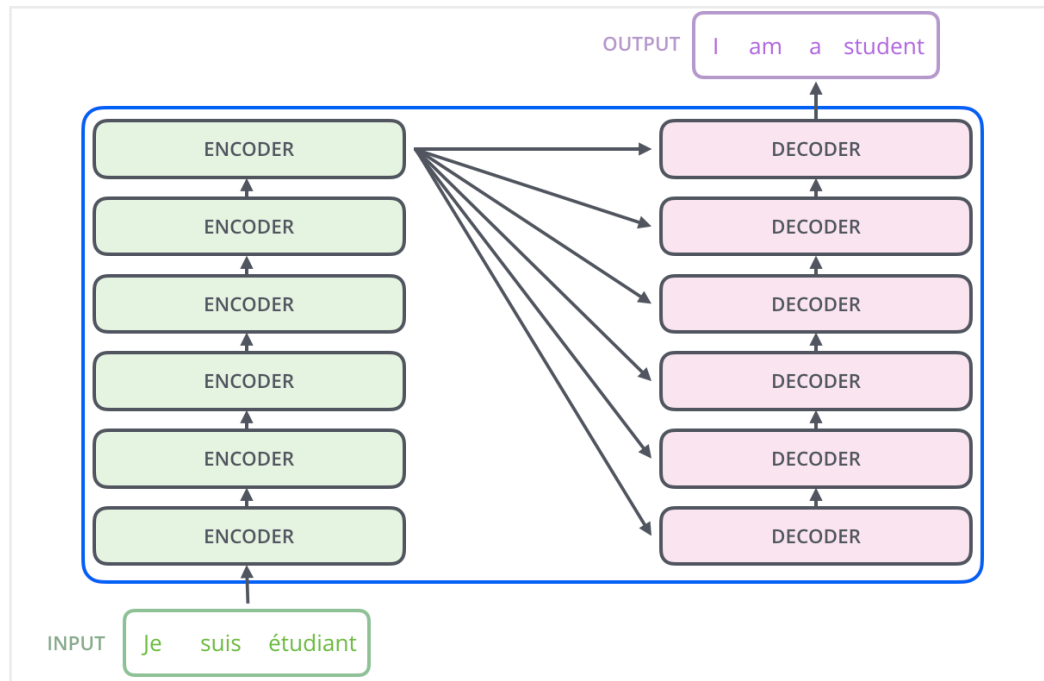


TRansformer

Attention is you need

<https://jalammar.github.io/illustrated-transformer/>

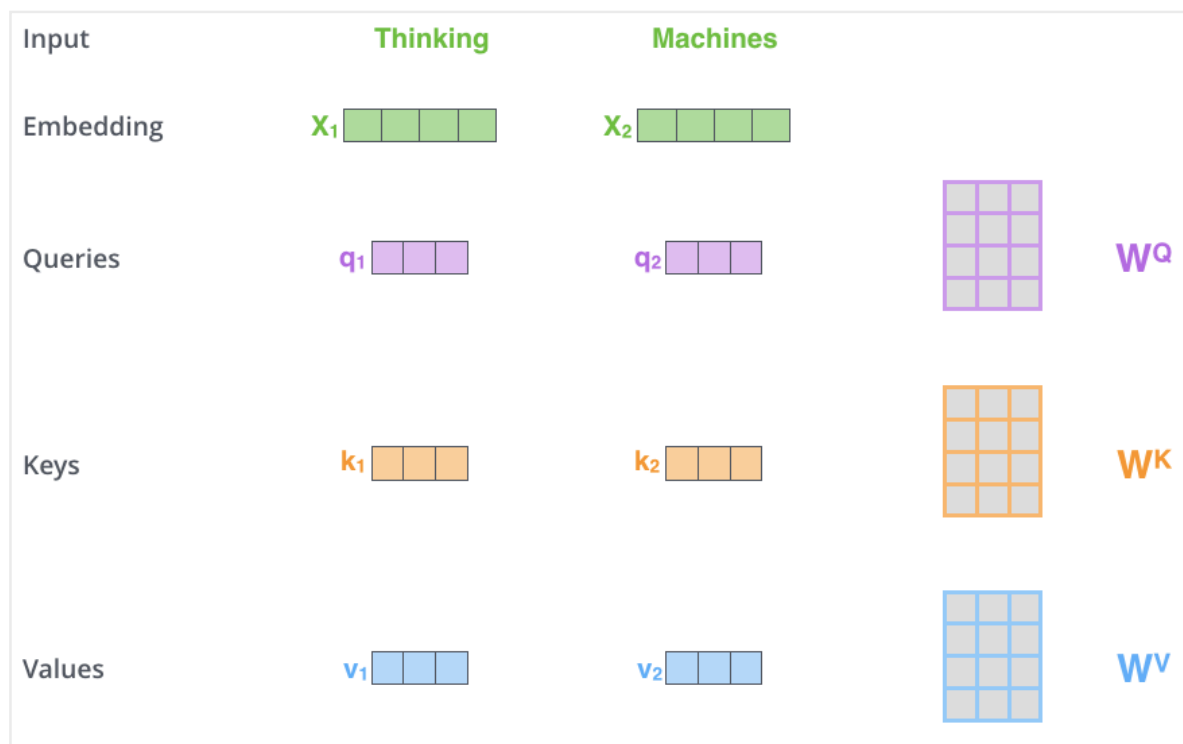


self-attention layer – a layer that helps the encoder look at other words in the input sentence as it encodes a specific word. We'll look closer at self-attention later in the post.



https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb

The **first step** in calculating self-attention is to create three vectors from each of the encoder's input vectors (in this case, the embedding of each word). So for each word, we create a Query vector, a Key vector, and a Value vector. These vectors are created by multiplying the embedding by three matrices that we trained during the training process.



What are the "query", "key", and "value" vectors?

They're abstractions that are useful for calculating and thinking about attention. Once you proceed with reading how attention is calculated below, you'll know pretty much all you need to know about the role each of these vectors plays.

The **second step** in calculating self-attention is to calculate a score. Say we're calculating the self-attention for the first word in this example, "Thinking". We need to score each word of the input sentence against this word. The score determines how much focus to place on other parts of the input sentence as we encode a word at a certain position.

The score is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring. So if we're processing the self-attention for the word in position #1, the first score would be the dot product of q_1 and k_1 . The second score would be the dot product of q_1 and k_2 .

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$

The **third and fourth steps** are to divide the scores by 8 (the square root of the dimension of the key vectors used in the paper – 64. This leads to having more stable gradients. There could be other possible values here, but this is the default), then pass the result through a softmax operation. Softmax normalizes the scores so they're all positive and add up to 1.

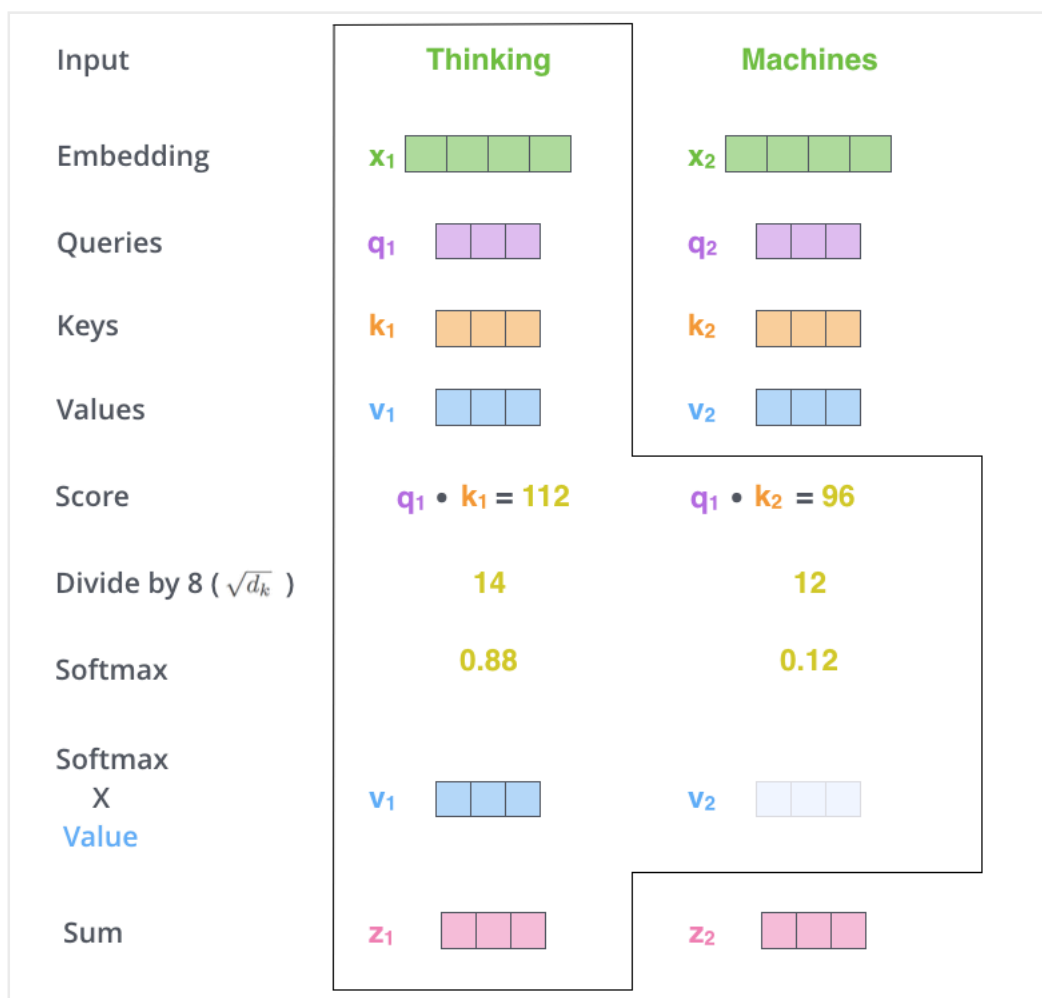
Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

This softmax score determines how much each word will be expressed at this position. Clearly the word at this position will have the highest softmax score,

but sometimes it's useful to attend to another word that is relevant to the current word.

The **fifth step** is to multiply each value vector by the softmax score (in preparation to sum them up). The intuition here is to keep intact the values of the word(s) we want to focus on, and down-out irrelevant words (by multiplying them by tiny numbers like 0.001, for example).

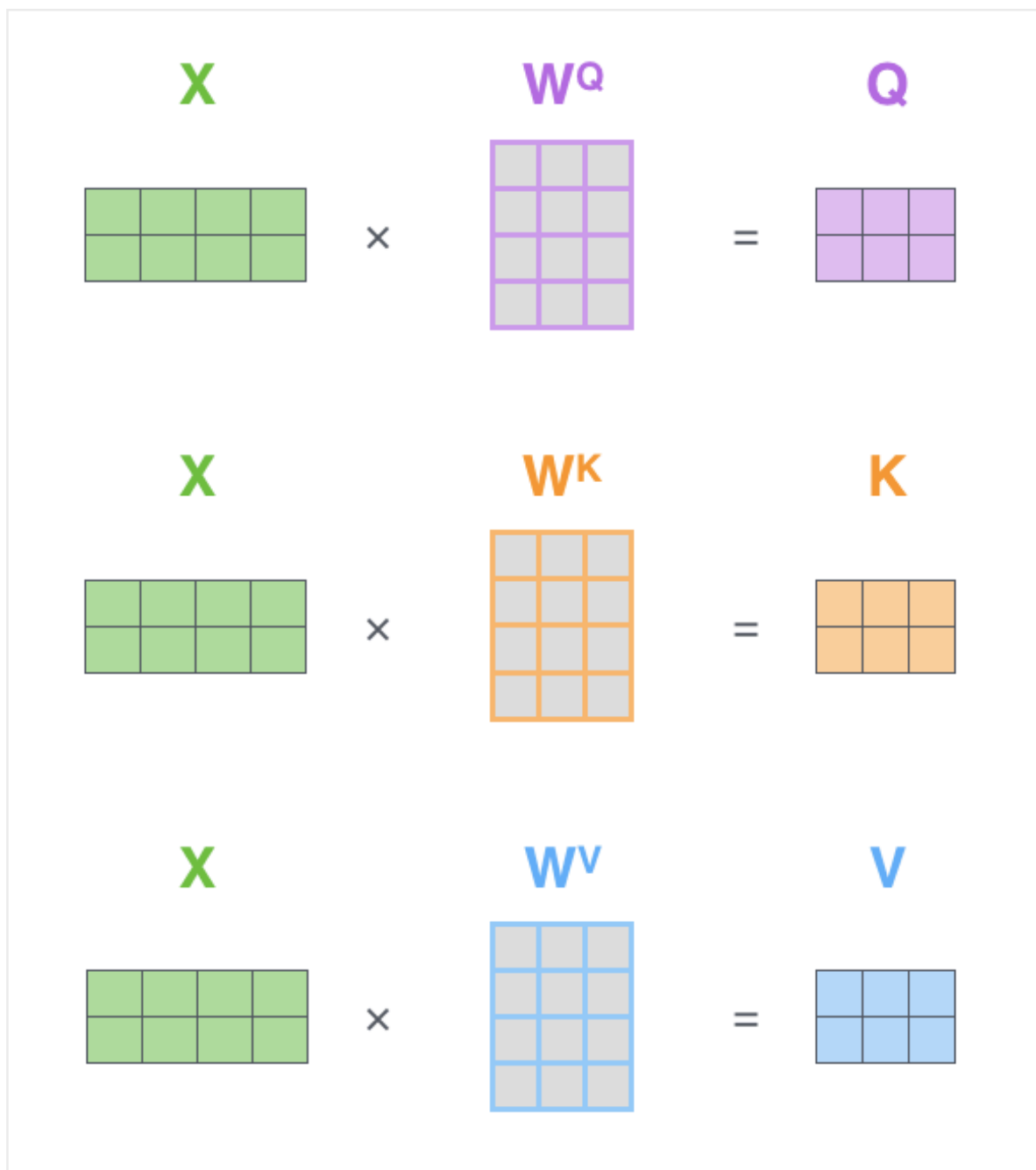
The **sixth step** is to sum up the weighted value vectors. This produces the output of the self-attention layer at this position (for the first word).



That concludes the self-attention calculation. The resulting vector is one we can send along to the feed-forward neural network. In the actual implementation, however, this calculation is done in matrix form for faster processing. So let's look at that now that we've seen the intuition of the calculation on the word level

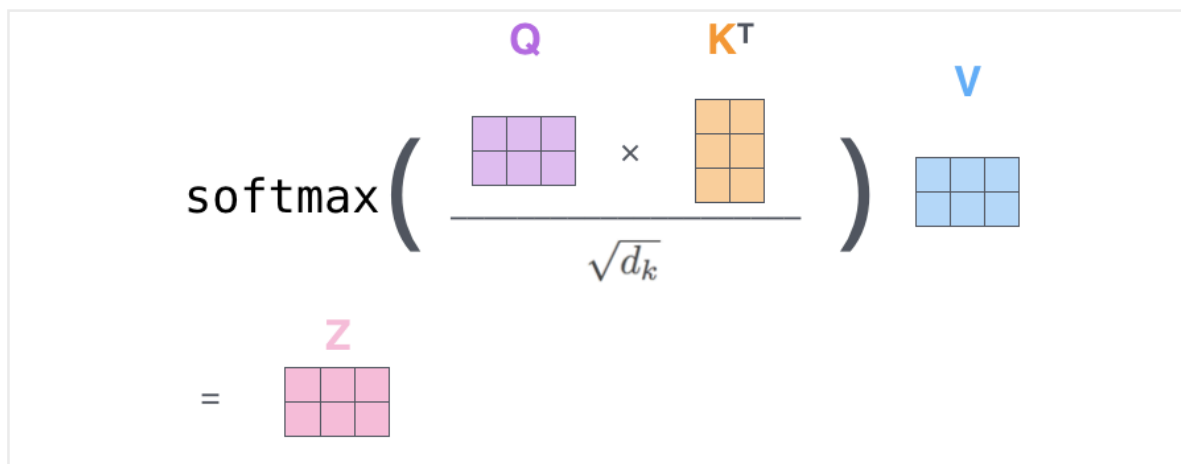
Matrix Calculation of Self-Attention

The first step is to calculate the Query, Key, and Value matrices. We do that by packing our embeddings into a matrix X , and multiplying it by the weight matrices we've trained (W_Q , W_K , W_V).



Every row in the X matrix corresponds to a word in the input sentence. We again see the difference in size of the embedding vector (512, or 4 boxes in the figure), and the $q/k/v$ vectors (64, or 3 boxes in the figure)

Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.

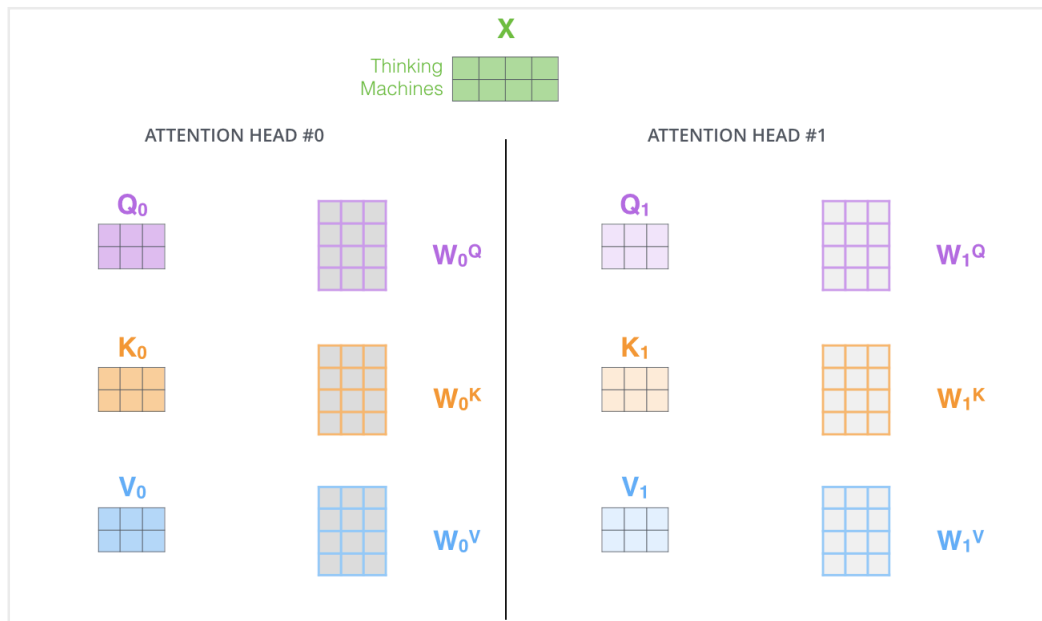


The self-attention calculation in matrix form

The Beast With Many Heads

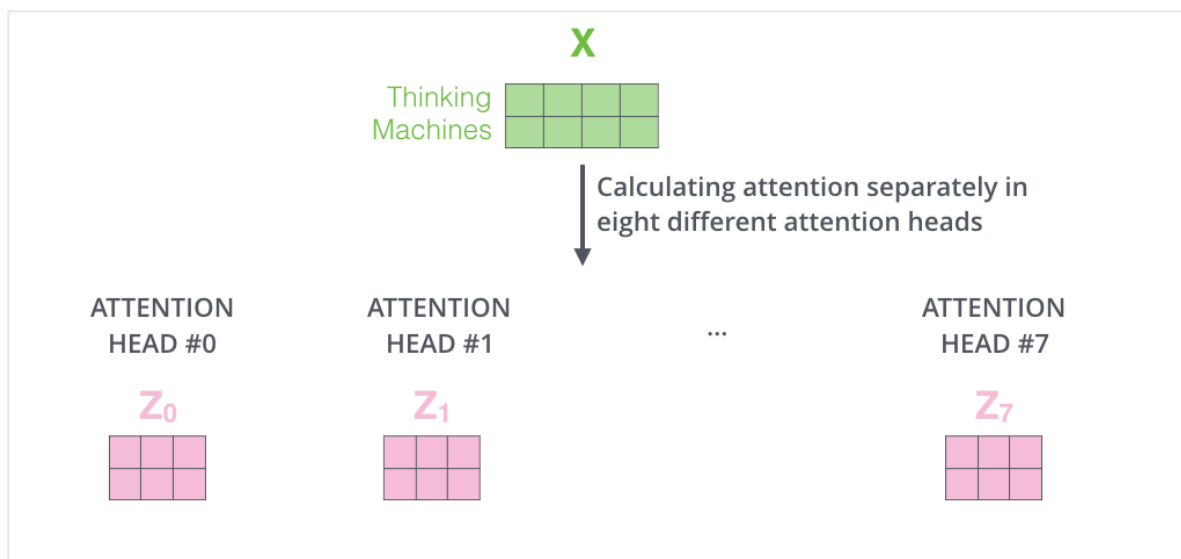
The paper further refined the self-attention layer by adding a mechanism called “multi-headed” attention. This improves the performance of the attention layer in two ways:

1. It expands the model's ability to focus on different positions. Yes, in the example above, **z1** contains a little bit of every other encoding, but it could be dominated by the actual word itself. If we're translating a sentence like “The animal didn't cross the street because it was too tired”, it would be useful to know which word “it” refers to.
2. It gives the attention layer multiple “representation subspaces”. As we'll see next, with multi-headed attention we have not only one, but multiple sets of Query/Key/Value weight matrices (the Transformer uses eight attention heads, so we end up with eight sets for each encoder/decoder). Each of these sets is randomly initialized. Then, after training, each set is used to project the input embeddings (or vectors from lower encoders/decoders) into a different representation subspace.

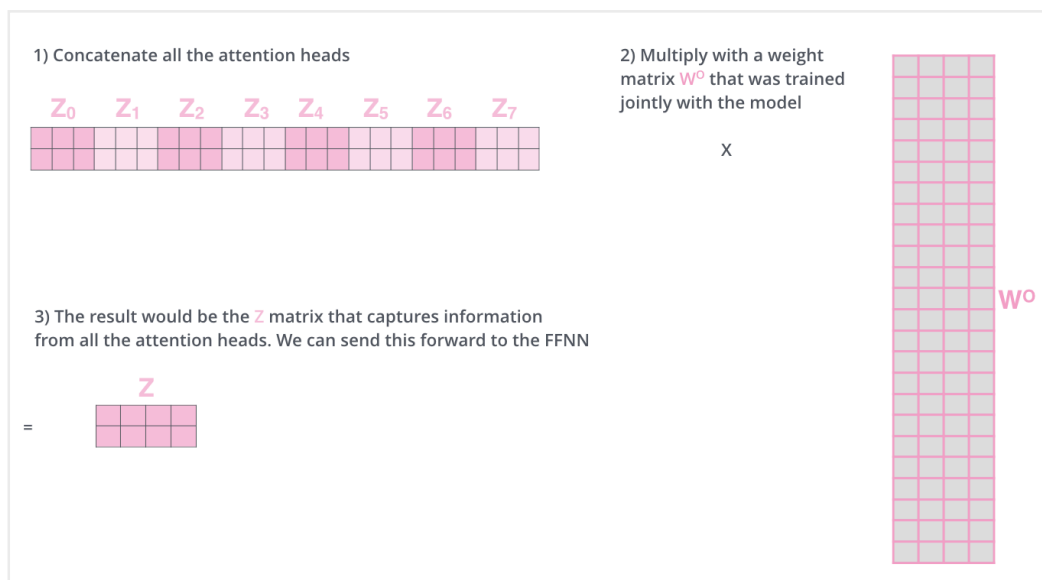


With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the $W_Q/W_K/W_V$ matrices to produce Q/K/V matrices.

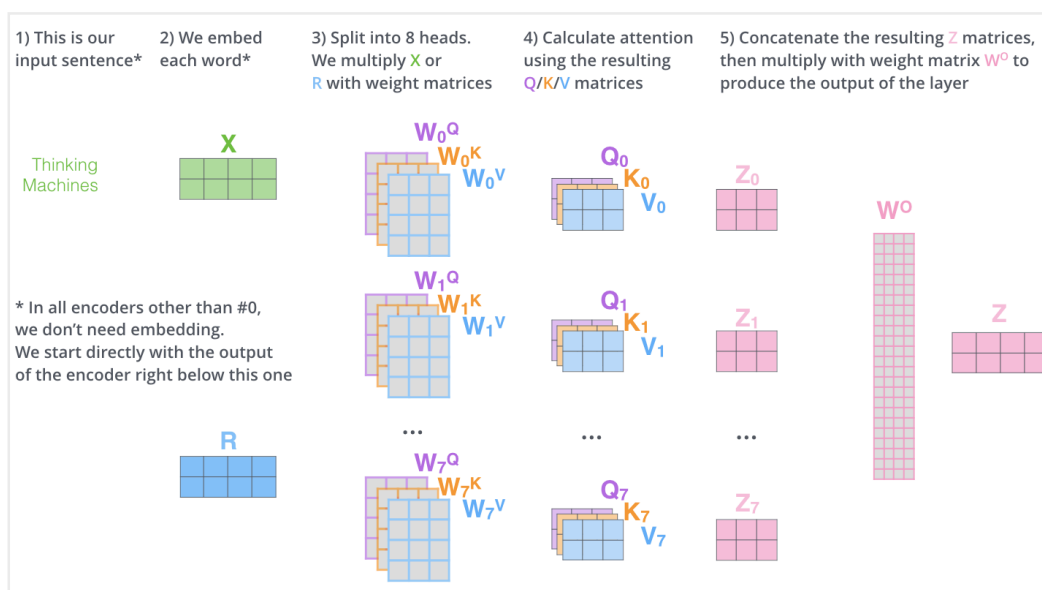
If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different Z matrices



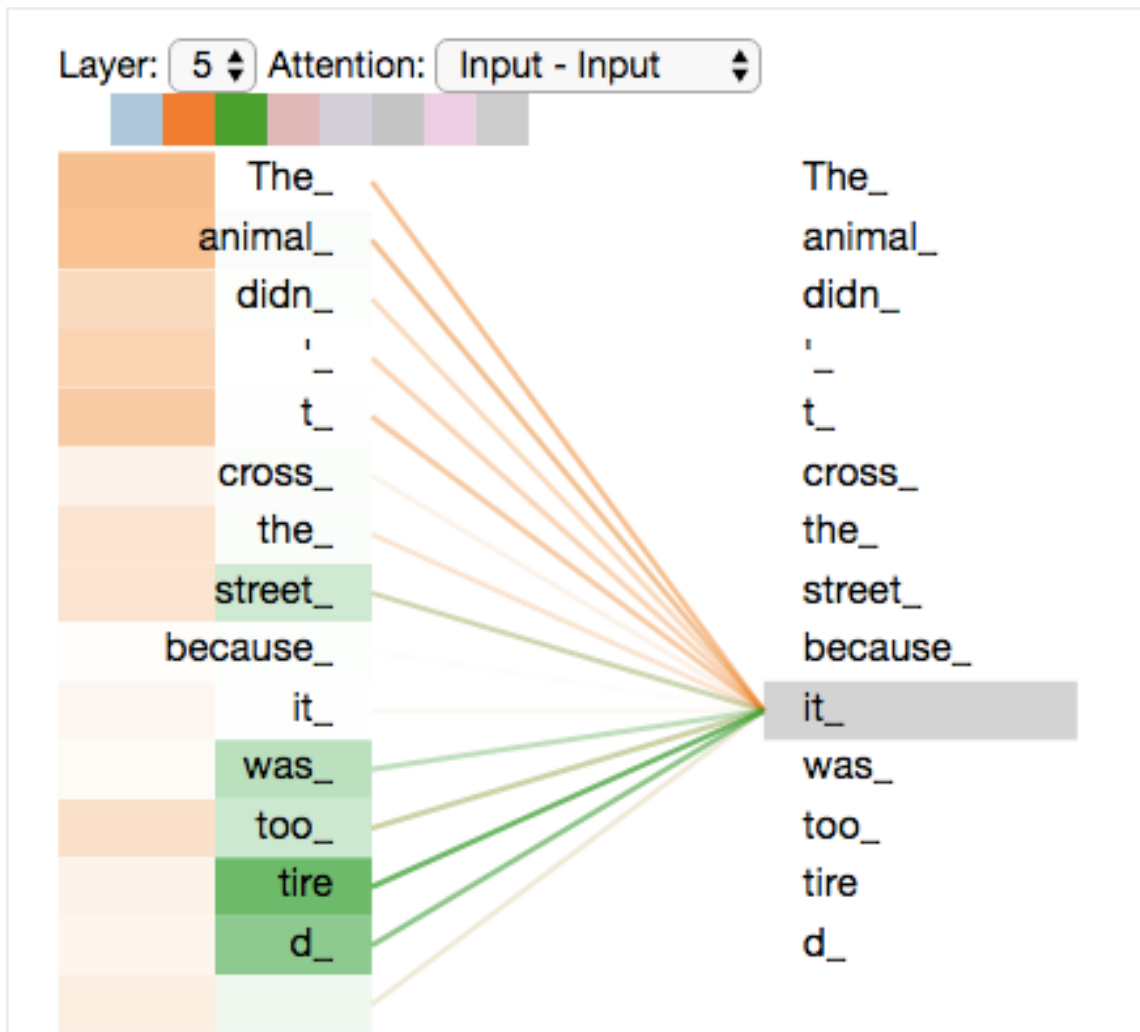
This leaves us with a bit of a challenge. The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix. How do we do that? We concat the matrices then multiply them by an additional weights matrix W_O .



That's pretty much all there is to multi-headed self-attention. It's quite a handful of matrices, I realize. Let me try to put them all in one visual so we can look at them in one place

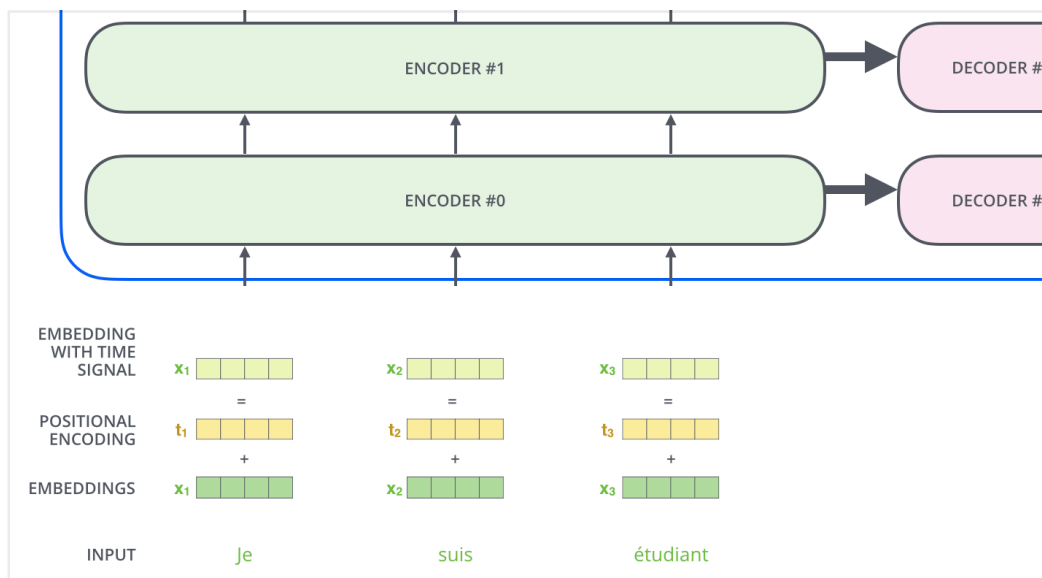


Now that we have touched upon attention heads, let's revisit our example from before to see where the different attention heads are focusing as we encode the word "it" in our example sentence:



As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".

If we add all the attention heads to the picture, however, things can be harder to interpret:



To give the model a sense of the order of the words, we add positional encoding vectors -- the values of which follow a specific pattern.

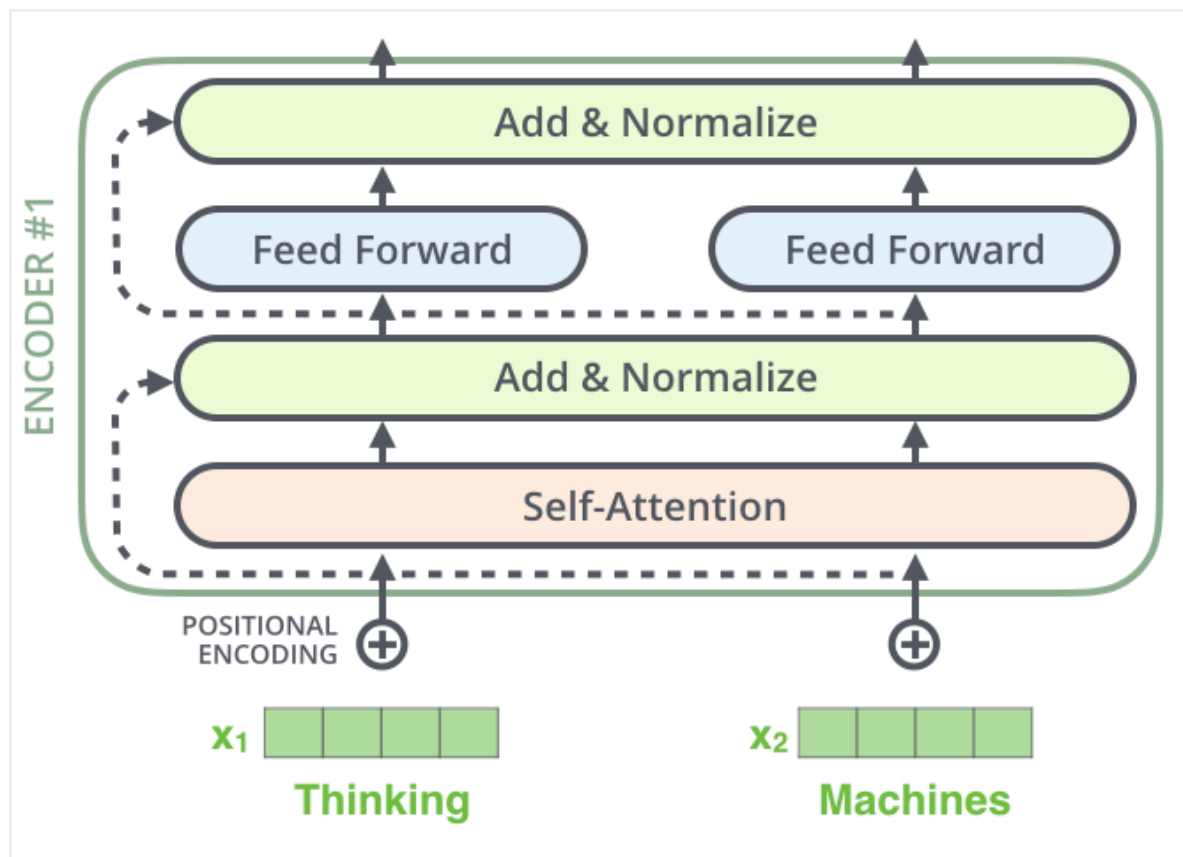
If we assumed the embedding has a dimensionality of 4, the actual positional encodings would look like this:

POSITIONAL ENCODING	0	0	1	1
	+			
EMBEDDINGS	x1	x2	x3	
INPUT	Je	suis	étudiant	

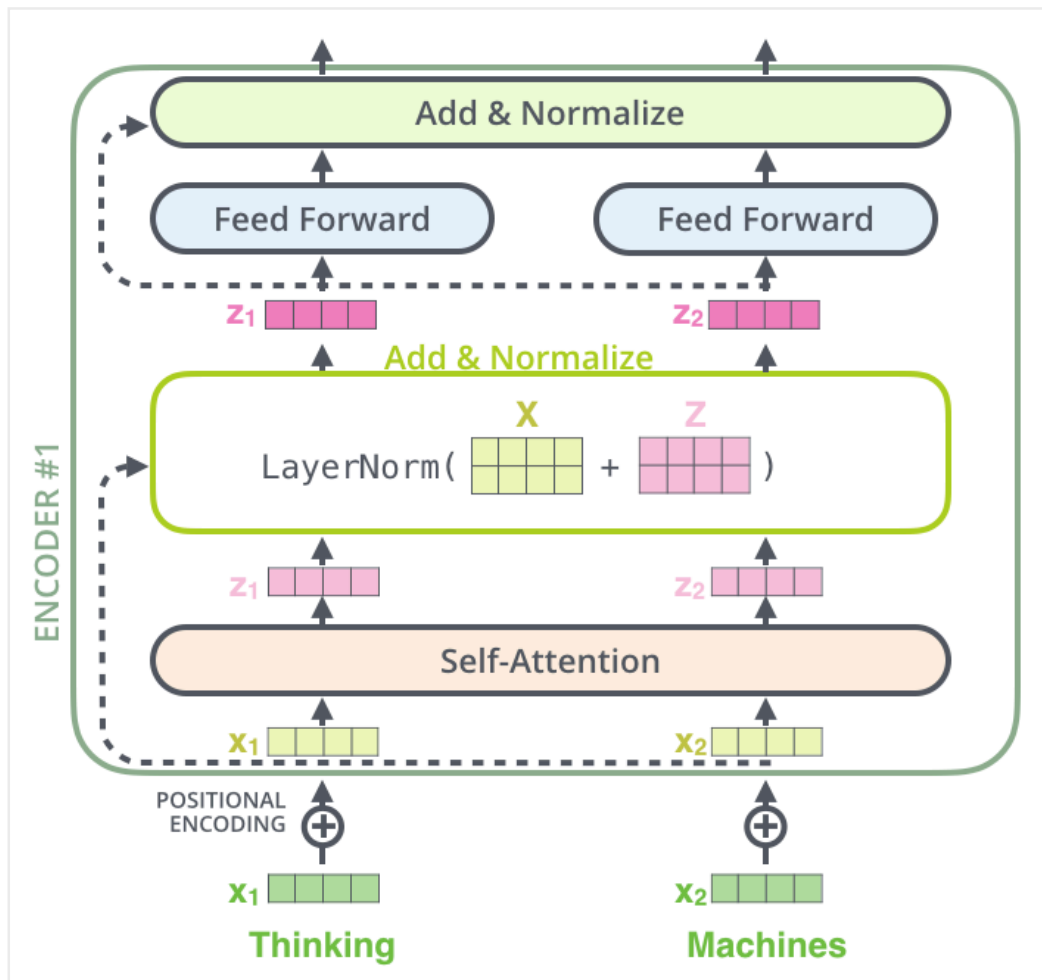
A real example of positional encoding with a toy embedding size of 4

The Residuals

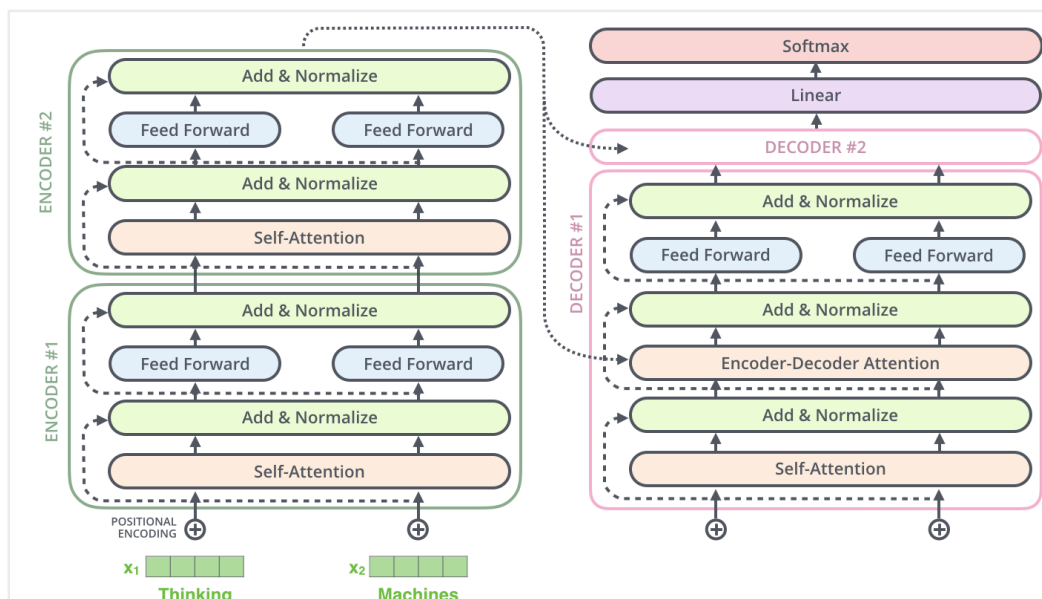
One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a **layer-normalization** step.



If we're to visualize the vectors and the layer-norm operation associated with self attention, it would look like this:



This goes for the sub-layers of the decoder as well. If we're to think of a Transformer of 2 stacked encoders and decoders, it would look something like this:

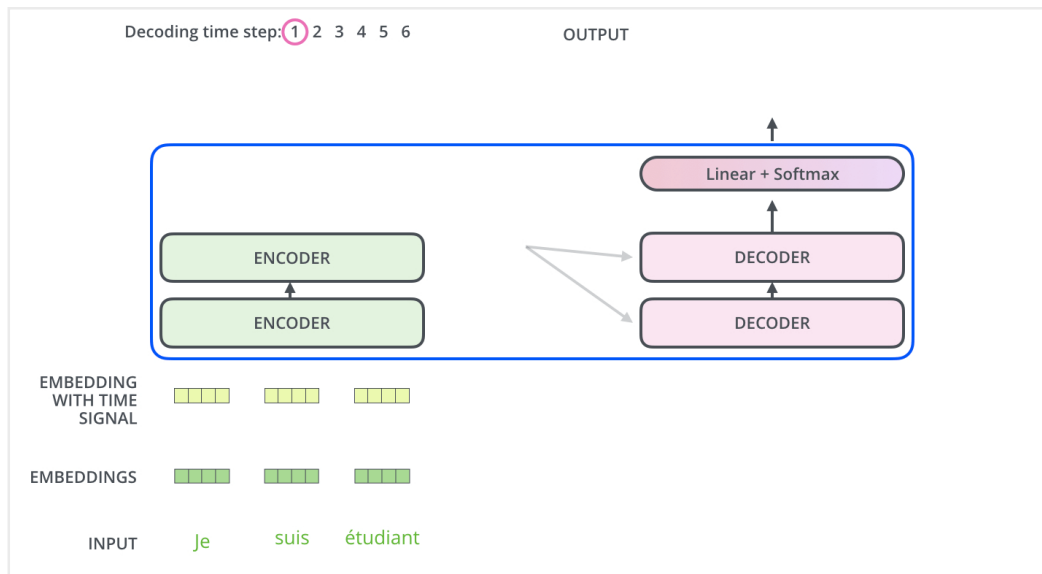


The Decoder Side

Now that we've covered most of the concepts on the encoder side, we basically

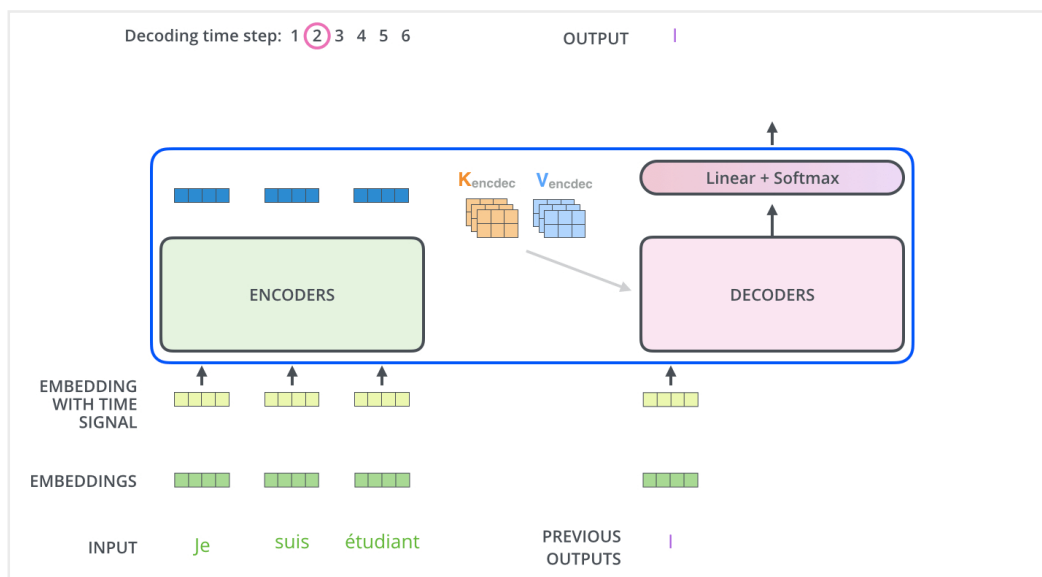
know how the components of decoders work as well. But let's take a look at how they work together.

The encoder starts by processing the input sequence. The output of the top encoder is then transformed into a set of attention vectors K and V . These are to be used by each decoder in its "encoder-decoder attention" layer which helps the decoder focus on appropriate places in the input sequence:



After finishing the encoding phase, we begin the decoding phase. Each step in the decoding phase outputs an element from the output sequence (the English translation sentence in this case).

The following steps repeat the process until a special symbol is reached indicating the transformer decoder has completed its output. The output of each step is fed to the bottom decoder in the next time step, and the decoders bubble up their decoding results just like the encoders did. And just like we did with the encoder inputs, we embed and add positional encoding to those decoder inputs to indicate the position of each word.



The self attention layers in the decoder operate in a slightly different way than the one in the encoder:

In the decoder, the self-attention layer is only allowed to attend to earlier

positions in the output sequence. This is done by masking future positions (setting them to -inf) before the softmax step in the self-attention calculation. The “Encoder-Decoder Attention” layer works just like multiheaded self-attention, except it creates its Queries matrix from the layer below it, and takes the Keys and Values matrix from the output of the encoder stack.

The Final Linear and Softmax Layer

The decoder stack outputs a vector of floats. How do we turn that into a word? That’s the job of the final Linear layer which is followed by a Softmax Layer. The Linear layer is a simple fully connected neural network that projects the vector produced by the stack of decoders, into a much, much larger vector called a logits vector.

Let’s assume that our model knows 10,000 unique English words (our model’s “output vocabulary”) that it’s learned from its training dataset. This would make the logits vector 10,000 cells wide – each cell corresponding to the score of a unique word. That is how we interpret the output of the model followed by the Linear layer.

The softmax layer then turns those scores into probabilities (all positive, all add up to 1.0). The cell with the highest probability is chosen, and the word associated with it is produced as the output for this time step

Complete variable table

Complete variable table (all variables)

Variable	Name	What it is	How it's determined	Example value
V	Vocabulary size	Total number of tokens the model knows	Chosen by researchers	50,000
S	Sequence length	Number of tokens in the input	Chosen by researchers	3
D	Embedding dimension	Length of each token's vector	Chosen by researchers	4
H	Number of heads	How many attention heads run in parallel	Chosen by researchers	2

d_k	Key/Query dimension	Size of Q and K vectors per head	Calculated: $D \div H$	$4 \div 2 = 2$
d_v	Value dimension	Size of V vectors per head	Calculated: $D \div H$	$4 \div 2 = 2$
d_{ff}	FFN hidden dimension	Size of expanded vector inside FFN	Calculated: $D \times 4$	$4 \times 4 = 16$
N	Number of blocks	How many Transformer blocks stack	Chosen by researchers	2
E	Embedding table	Matrix storing all token vectors	Learned during training	$[50,000 \times 4]$
W_Q	Query weight matrix	Projects embedding into Q space	Learned during training	$[4 \times 2]$
W_K	Key weight matrix	Projects embedding into K space	Learned during training	$[4 \times 2]$
W_V	Value weight matrix	Projects embedding into V space	Learned during training	$[4 \times 2]$
Q	Query matrix	"What am I looking for?" — per token	Calculated: $E \times W_Q$	$[S \times d_k] = [3 \times 2]$
K	Key matrix	"What do I offer?" — per token	Calculated: $E \times W_K$	$[S \times d_k] = [3 \times 2]$
V	Value matrix	"What info do I carry?" — per token	Calculated: $E \times W_V$	$[S \times d_v] = [3 \times 2]$
A	Attention matrix	Attention weights between all token pairs	Calculated: $\text{softmax}(Q \cdot K^T \div \sqrt{d_k})$	$[S \times S] = [3 \times 3]$

W_O	Output weight matrix	Projects concatenated heads back to D	Learned during training	$[D \times D] = [4 \times 4]$
W₁	FFN first layer	Expands dimension inside FFN	Learned during training	$[D \times d_{\text{ff}}] = [4 \times 16]$
W₂	FFN second layer	Compresses back to D	Learned during training	$[d_{\text{ff}} \times D] = [16 \times 4]$
b₁	FFN first bias	Bias added after W ₁	Learned during training	$[d_{\text{ff}}] = [16]$
b₂	FFN second bias	Bias added after W ₂	Learned during training	$[D] = [4]$
W_{out}	Output projection	Projects final vector to vocabulary scores	Learned during training	$[D \times V] = [4 \times 50,000]$
lr	Learning rate	Step size during weight updates	Chosen by researchers	0.001
Loss	Cross entropy loss	How wrong the prediction was	Calculated during training	scalar number
∇	Gradient	Direction and magnitude to adjust each weight	Calculated via backpropagation	same shape as each matrix

How these choices affect the model

If you increase...	Effect	Cost
D	Richer token representations	More memory, more compute
H	More types of relationships captured	More compute

N	Deeper reasoning, more abstraction	Much more compute
S	Longer context window	Quadratic cost — doubling S quadruples attention compute
V	More expressive vocabulary	More memory for embedding table

Part 1 — Training example

We will use tiny numbers so every calculation is visible.

Hyperparameters:

$S = 3$, $D = 4$, $H = 2$, $d_k = 2$, $d_v = 2$, $d_{ff} = 16$, $N = 1$

Training sentence:

Input: "The cat sat"

Expected: "on" (token index = 8 in our tiny vocabulary)

Step 1 — Tokenize

"The" → index 1

"cat" → index 2

"sat" → index 3

Tokens = [1, 2, 3] shape: [S] = [3]

Step 2 — Embedding lookup

Embedding table E shape: $[V \times D] = [50,000 \times 4]$

We look up rows 1, 2, 3:

$e1(\text{"The"}) = [0.1, 0.2, -0.3, 0.5]$

$e2(\text{"cat"}) = [0.2, -0.5, 0.8, 0.1]$

$e3(\text{"sat"}) = [0.5, 0.1, 0.2, -0.4]$

E_{input} shape: $[S \times D] = [3 \times 4]$

Step 3 — Create Q, K, V (Head 1)

Initial random W_Q , W_K , W_V each shape $[D \times d_k] = [4 \times 2]$:

$W_Q = \begin{bmatrix} 0.1 & 0.2 \\ -0.3 & 0.4 \end{bmatrix}$ $W_K = \begin{bmatrix} 0.3 & -0.1 \\ 0.2 & 0.5 \end{bmatrix}$ $W_V = \begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.3 \end{bmatrix}$

$\begin{bmatrix} 0.5, & -0.2 \\ 0.1, & 0.3 \end{bmatrix}$	$\begin{bmatrix} -0.4, & 0.1 \\ 0.1, & 0.2 \end{bmatrix}$	$\begin{bmatrix} 0.4, & -0.2 \\ 0.1, & 0.5 \end{bmatrix}$
---	---	---

Calculate $Q = E_{\text{input}} \times W_Q$:

$$\begin{aligned} Q1(\text{"The"}) &= [0.1 \times 0.1 + 0.2 \times (-0.3) + (-0.3) \times 0.5 + 0.5 \times 0.1, \\ &\quad 0.1 \times 0.2 + 0.2 \times 0.4 + (-0.3) \times (-0.2) + 0.5 \times 0.3] \\ &= [0.01 - 0.06 - 0.15 + 0.05, \\ &\quad 0.02 + 0.08 + 0.06 + 0.15] \\ &= [-0.15, 0.31] \end{aligned}$$

$$Q2(\text{"cat"}) = [0.09, 0.21]$$

$$Q3(\text{"sat"}) = [0.18, -0.14]$$

Q shape: $[3 \times 2]$

Same process for K and V:

$$K1(\text{"The"}) = [0.12, -0.08]$$

$$K2(\text{"cat"}) = [-0.21, 0.34]$$

$$K3(\text{"sat"}) = [0.15, 0.07]$$

$$V1(\text{"The"}) = [0.11, 0.28]$$

$$V2(\text{"cat"}) = [0.31, -0.14]$$

$$V3(\text{"sat"}) = [0.22, 0.18]$$

Step 4 — Attention scores

Calculate $Q \times K^T$:

$Q \times K^T$ shape: $[3 \times 2] \times [2 \times 3] = [3 \times 3]$

Each cell = dot product of one Q row with one K row:

$$A_{\text{row}}[1,1] = Q1 \cdot K1 = (-0.15 \times 0.12) + (0.31 \times -0.08) = -0.018 - 0.025 = -0.043$$

$$A_{\text{row}}[1,2] = Q1 \cdot K2 = (-0.15 \times -0.21) + (0.31 \times 0.34) = 0.032 + 0.105 = 0.137$$

$$A_{\text{row}}[1,3] = Q1 \cdot K3 = (-0.15 \times 0.15) + (0.31 \times 0.07) = -0.023 + 0.022 = -0.001$$

$$A_{\text{row}}[2,1] = Q2 \cdot K1 = (0.09 \times 0.12) + (0.21 \times -0.08) = 0.011 - 0.017 = -0.006$$

$$A_{\text{row}}[2,2] = Q2 \cdot K2 = (0.09 \times -0.21) + (0.21 \times 0.34) = -0.019 + 0.071 = 0.052$$

$$A_{\text{row}}[2,3] = Q2 \cdot K3 = (0.09 \times 0.15) + (0.21 \times 0.07) = 0.014 + 0.015 = 0.029$$

$$A_{\text{row}}[3,1] = Q3 \cdot K1 = (0.18 \times 0.12) + (-0.14 \times -0.08) = 0.022 + 0.011 = 0.033$$

$$A_{\text{row}}[3,2] = Q3 \cdot K2 = (0.18 \times -0.21) + (-0.14 \times 0.34) = -0.038 - 0.048 = -0.086$$

$$A_{\text{row}}[3,3] = Q3 \cdot K3 = (0.18 \times 0.15) + (-0.14 \times 0.07) = 0.027 - 0.010 = 0.017$$

$$A_{\text{row}} = [[-0.043, 0.137, -0.001]$$

$[-0.006, 0.052, 0.029]$
 $[0.033, -0.086, 0.017]$

Step 5 — Scale by $\sqrt{d_k}$

$$\sqrt{d_k} = \sqrt{2} = 1.414$$

$$A_{\text{scaled}} = A_{\text{raw}} \div 1.414$$

$A_{\text{scaled}} = [[-0.030, 0.097, -0.001]$
 $[-0.004, 0.037, 0.021]$
 $[0.023, -0.061, 0.012]]$

Step 6 — Softmax per row

$\text{softmax}(x_i) = e^{x_i} \div \sum(e^{x_j})$
Row 1: $[-0.030, 0.097, -0.001]$

$e^{-0.030} = 0.970$
 $e^{0.097} = 1.102$
 $e^{-0.001} = 0.999$
sum = 3.071

$A[1] = [0.970/3.071, 1.102/3.071, 0.999/3.071]$
= $[0.316, 0.359, 0.325]$ ← adds up to 1.0 
Row 2: $[-0.004, 0.037, 0.021]$

$e^{-0.004} = 0.996$
 $e^{0.037} = 1.038$
 $e^{0.021} = 1.021$
sum = 3.055

$A[2] = [0.326, 0.340, 0.334]$ ← adds up to 1.0 
Row 3: $[0.023, -0.061, 0.012]$

$e^{0.023} = 1.023$
 $e^{-0.061} = 0.941$
 $e^{0.012} = 1.012$
sum = 2.976

$A[3] = [0.344, 0.316, 0.340]$ ← adds up to 1.0 
Final attention matrix:

A = [[0.316, 0.359, 0.325]
[0.326, 0.340, 0.334]
[0.344, 0.316, 0.340]]

Step 7 — Weighted sum of V

Output = A × V, shape: [3 × 3] × [3 × 2] = [3 × 2]

Out1("The") = 0.316×V1 + 0.359×V2 + 0.325×V3
= 0.316×[0.11, 0.28] + 0.359×[0.31,-0.14] + 0.325×[0.22, 0.18]
= [0.035, 0.088] + [0.111,-0.050] + [0.072, 0.059]
= [0.218, 0.097]

Out2("cat") = 0.326×[0.11,0.28] + 0.340×[0.31,-0.14] + 0.334×[0.22,0.18]
= [0.036,0.091] + [0.105,-0.048] + [0.073,0.060]
= [0.214, 0.103]

Out3("sat") = 0.344×[0.11,0.28] + 0.316×[0.31,-0.14] + 0.340×[0.22,0.18]
= [0.038,0.096] + [0.098,-0.044] + [0.075,0.061]
= [0.211, 0.113]

Head1_output shape: [3 × 2]

Step 8 — Multi-head concatenation

We have 2 heads. Head 2 runs the same process with its own W_Q2, W_K2, W_V2 (different random initialization). Let's say Head 2 produces:

Head2_output = [[0.15, 0.09],
[0.12, 0.11],
[0.18, 0.07]] shape: [3 × 2]

Concatenate Head 1 and Head 2:

Concat = [[0.218, 0.097, 0.15, 0.09],
[0.214, 0.103, 0.12, 0.11],
[0.211, 0.113, 0.18, 0.07]] shape: [3 × 4] = [S × D] ✓

Then multiply by W_O [D × D] = [4 × 4] to mix the heads:

Attention_output = Concat × W_O shape: [3 × 4]

Step 9 — Add & Normalize

Residual connection (add original embedding back):

$X = \text{Attention_output} + E_input$ shape: $[3 \times 4]$

Layer normalization (keeps values stable, not going to expand here to keep focus):

$X_norm = \text{LayerNorm}(X)$ shape: $[3 \times 4]$

Step 10 — FFN

$W1$ shape: $[4 \times 16]$, $b1$ shape: $[16]$

$W2$ shape: $[16 \times 4]$, $b2$ shape: $[4]$

For each token independently:

$h = \text{GELU}(X_norm \times W1 + b1)$ shape: $[3 \times 16]$

$\text{FFN_output} = h \times W2 + b2$ shape: $[3 \times 4]$

Add & Normalize again:

$X_final = \text{LayerNorm}(\text{FFN_output} + X_norm)$ shape: $[3 \times 4]$

This completes **one Transformer block**. With $N=2$ this whole process repeats once more.

Step 11 — Output projection

Take only the **last token** vector ("sat"):

$last = X_final[3]$ shape: $[1 \times 4]$

Multiply by W_out $[4 \times 50,000]$:

$scores = last \times W_out$ shape: $[1 \times 50,000]$

Apply softmax:

$probs = \text{softmax}(scores)$ shape: $[1 \times 50,000]$

With random weights the result might be:

"on" → 0.00003 ← correct answer, but nearly zero probability

"dog" → 0.00821

"pizza" → 0.00614

...

Step 12 — Loss

correct token = "on" = index 8

probability assigned to "on" = 0.00003

Loss = $-\log(0.00003) = 10.41$ ← very high, model is very wrong

Step 13 — Backpropagation

Calculate gradient of Loss with respect to **every weight**:

∇W_{out} shape: $[4 \times 50,000]$

∇W_2 shape: $[16 \times 4]$

∇W_1 shape: $[4 \times 16]$

∇W_O shape: $[4 \times 4]$

∇W_V shape: $[4 \times 2]$

∇W_K shape: $[4 \times 2]$

∇W_Q shape: $[4 \times 2]$

∇E shape: $[50,000 \times 4]$

Step 14 — Gradient descent

Update every weight:

$W_Q = W_Q - lr \times \nabla W_Q$

$W_K = W_K - lr \times \nabla W_K$

$W_V = W_V - lr \times \nabla W_V$

...all matrices updated...

Step 15 — Repeat

Next training example runs the same pipeline with the **updated weights**. After billions of steps the loss gets very small and the model predicts correctly.

Part 2 — Inference example

Inference is **exactly the same as the forward pass in training** — Steps 1 through 11 — but with two key differences:

1. Weights are FROZEN — no backpropagation, no updates
2. We generate token by token in a loop

Inference loop: "The cat sat" → generate response

Iteration 1:

Input: ["The", "cat", "sat"]
 ↓ full forward pass (steps 1-11)
 ↓ softmax over 50,000 tokens
 Output: "on" (highest probability)

Iteration 2:

Input: ["The", "cat", "sat", "on"] ← append predicted token
 ↓ full forward pass
 Output: "the"

Iteration 3:

Input: ["The", "cat", "sat", "on", "the"]
 ↓ full forward pass
 Output: "mat"

Iteration 4:

Input: ["The", "cat", "sat", "on", "the", "mat"]
 ↓ full forward pass
 Output: "<END>" ← special end token, stop generating
 Final output: "The cat sat on the mat" ✓

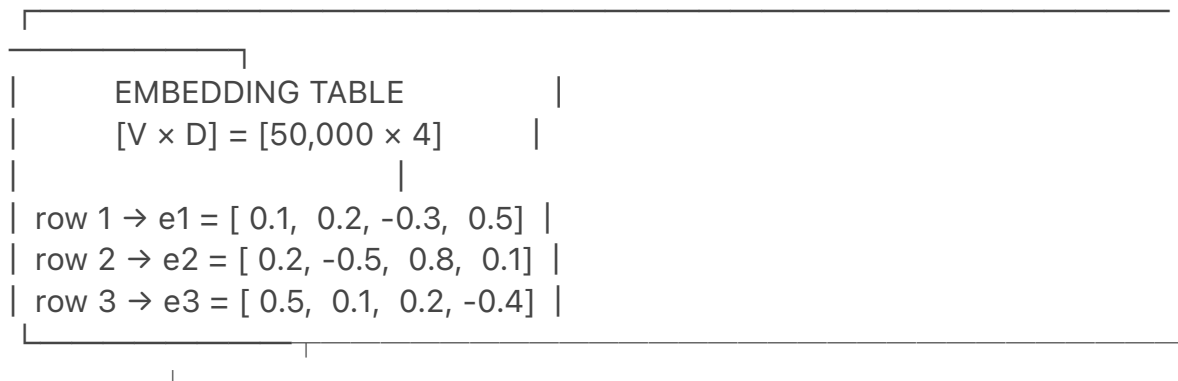
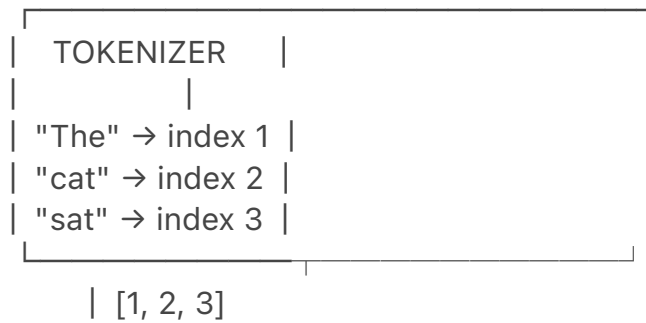
Training vs Inference — side by side

Step	Training	Inference
Tokenize	✓	✓
Embedding lookup	✓	✓
Q, K, V calculation	✓	✓
Attention	✓	✓
FFN	✓	✓
Output projection	✓	✓
Softmax	✓	✓
Calculate Loss	✓	✗
Backpropagation	✓	✗
Update weights	✓	✗
Loop token by token	✗	✓
Weights state	Being learned	Frozen

Does this give you the complete picture with no missing steps? 🎯

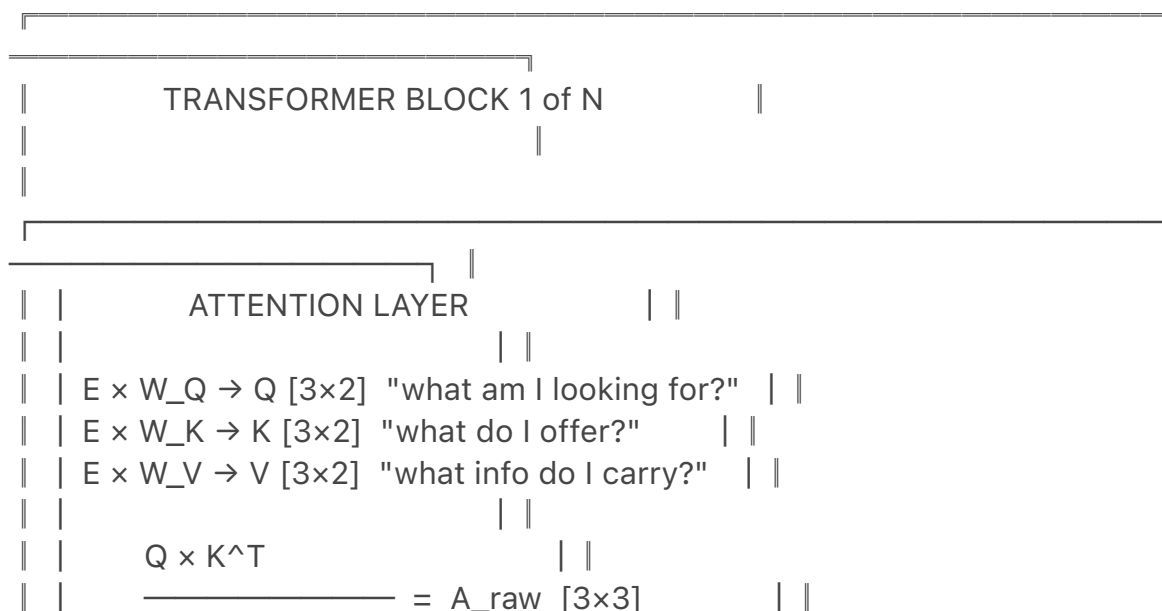
INPUT TEXT

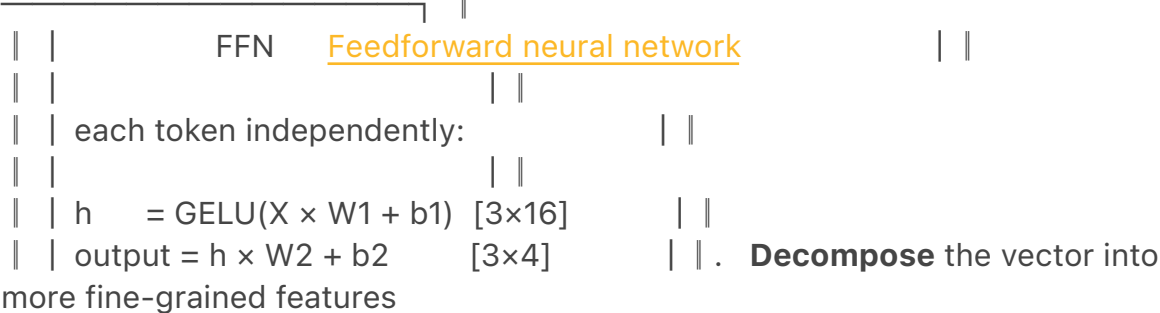
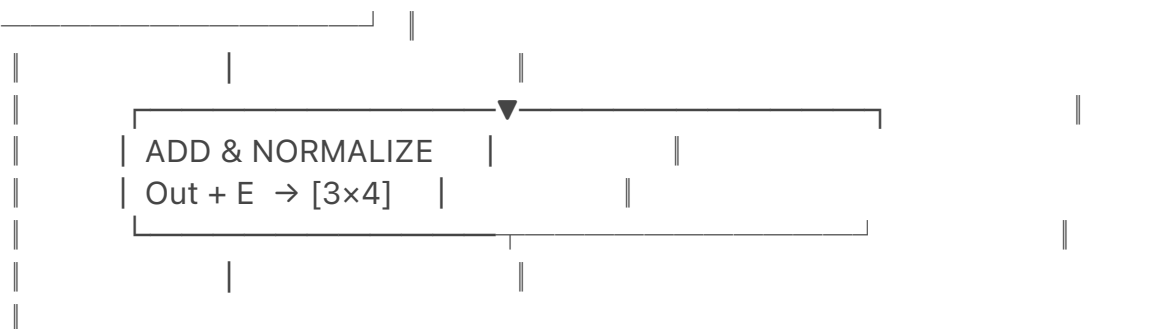
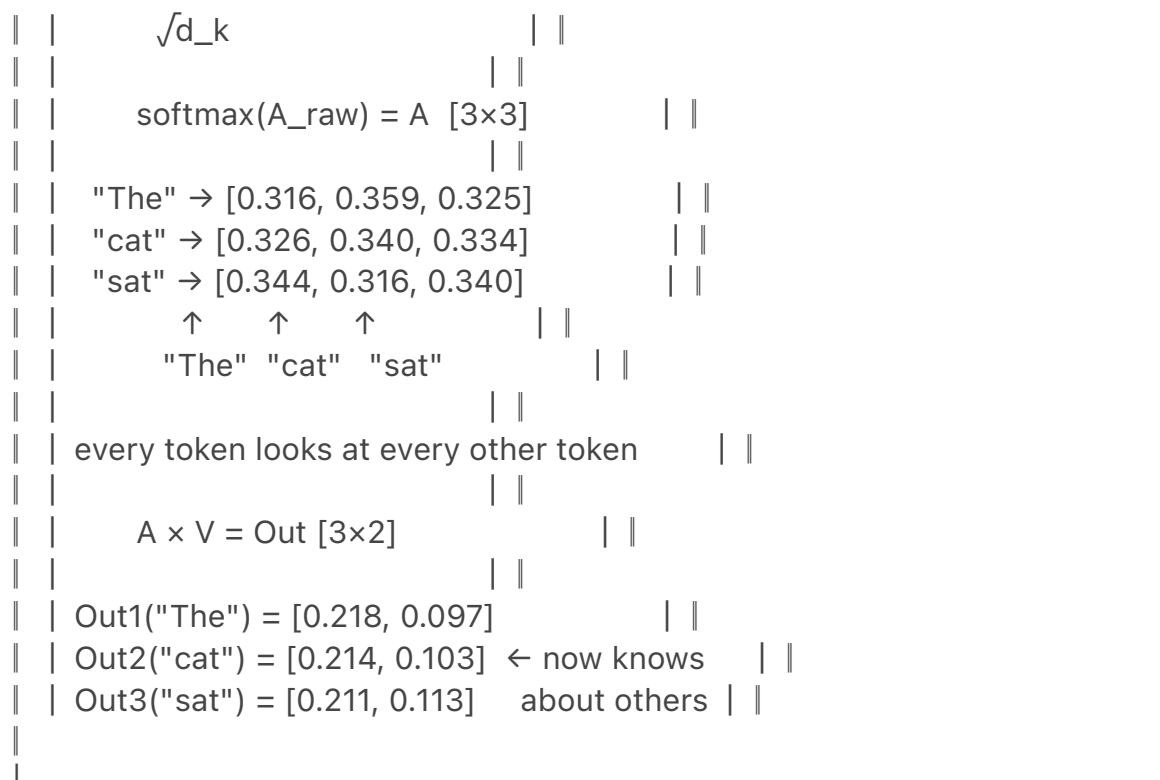
"The cat sat"



$E = [3 \times 4] \leftarrow$ static, no context yet

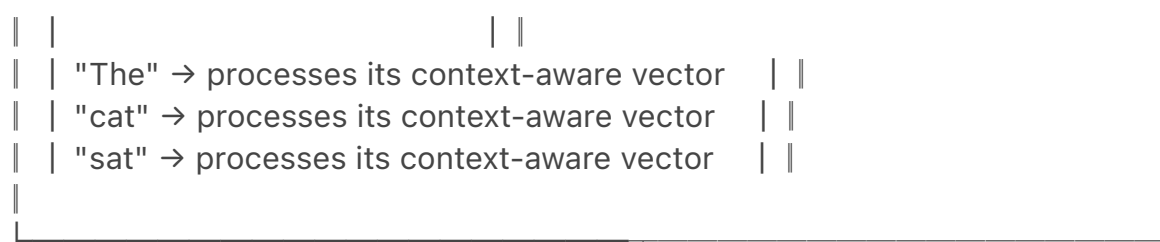
each token has same vector
regardless of surrounding words

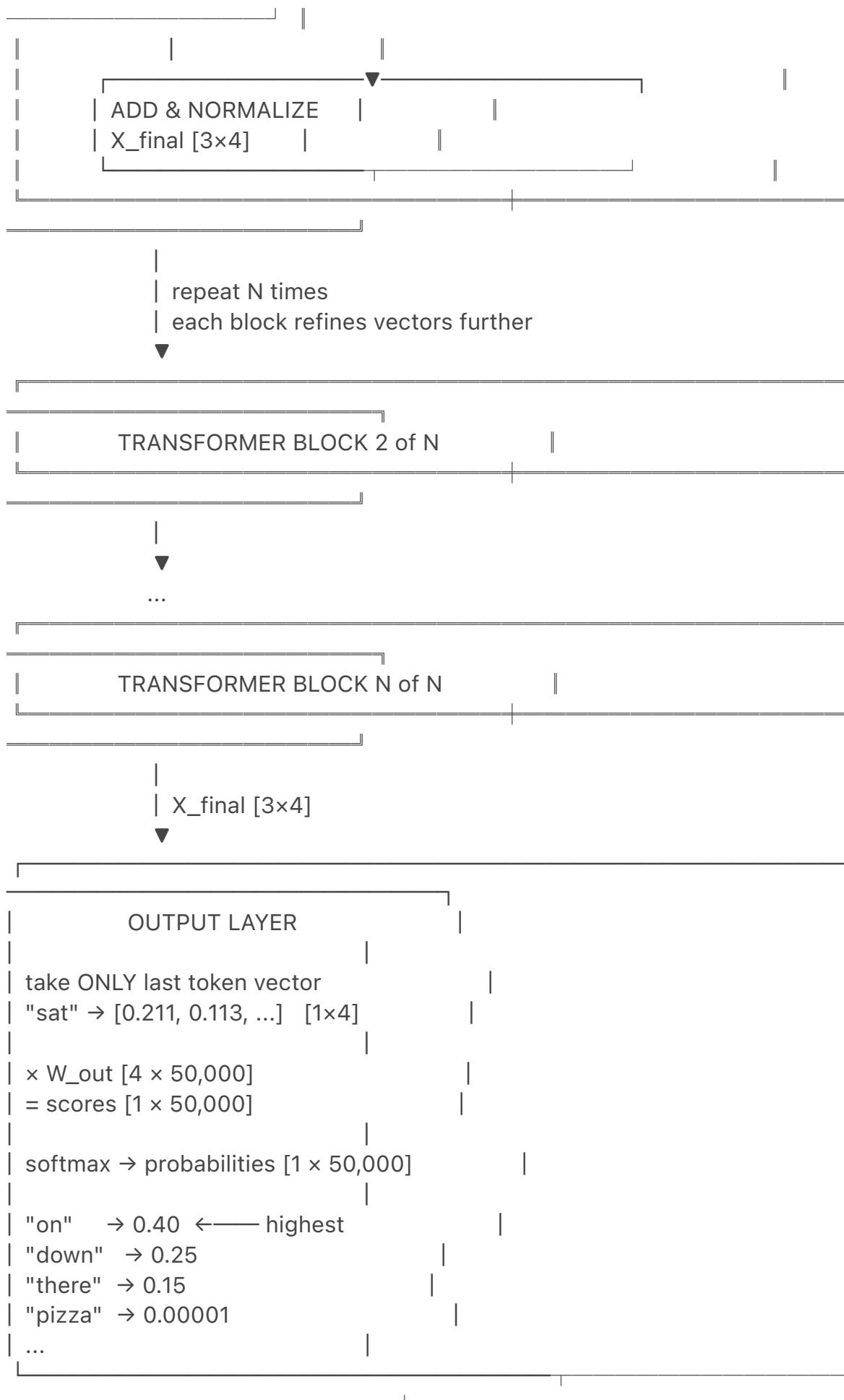


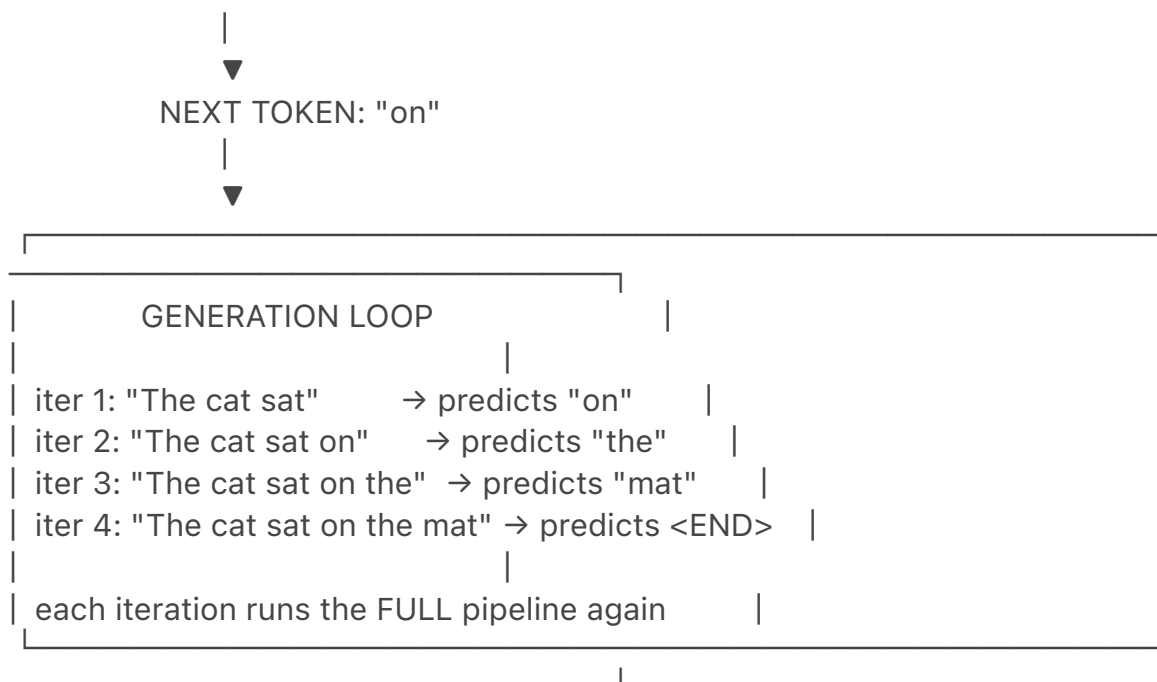


Apply non-linear transformations that would be impossible in 4 dimensions

Store and retrieve knowledge learned during training





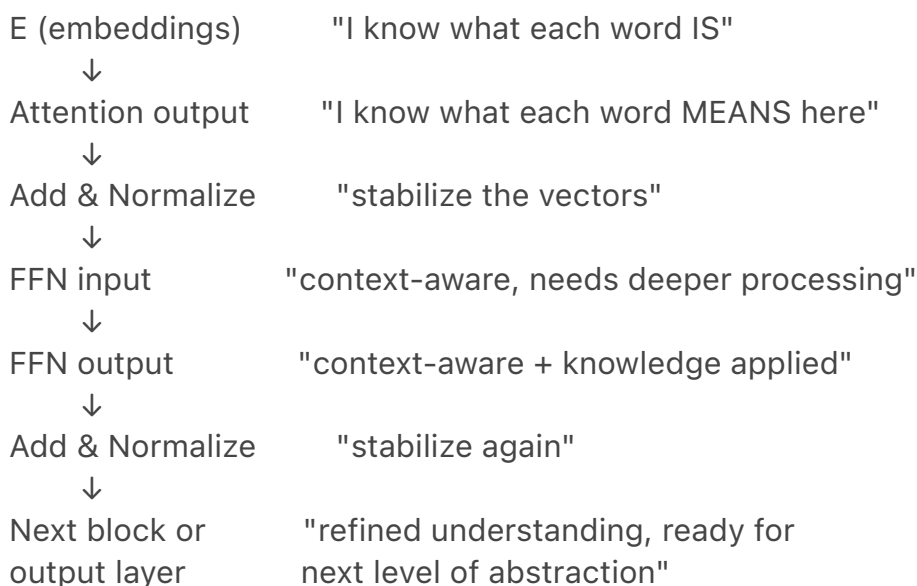


WITHOUT attention:

"bank" in any sentence → same vector $[-0.3, 0.9, -0.1, 0.6]$
 → model cannot distinguish river bank vs financial bank

WITH attention:

"bank" + "fishing" nearby → vector shifts toward river meaning
 "bank" + "deposit" nearby → vector shifts toward financial meaning
 → same word, different context, different vector



ENCODER ONLY	DECODER ONLY	ENCODER-DECODER (original)
--------------	--------------	----------------------------

Attention:	Bidirectional	Causal	Bidirectional +
------------	---------------	--------	-----------------

(sees all)	(sees past only)	Causal + Cross-attention
------------	------------------	--------------------------

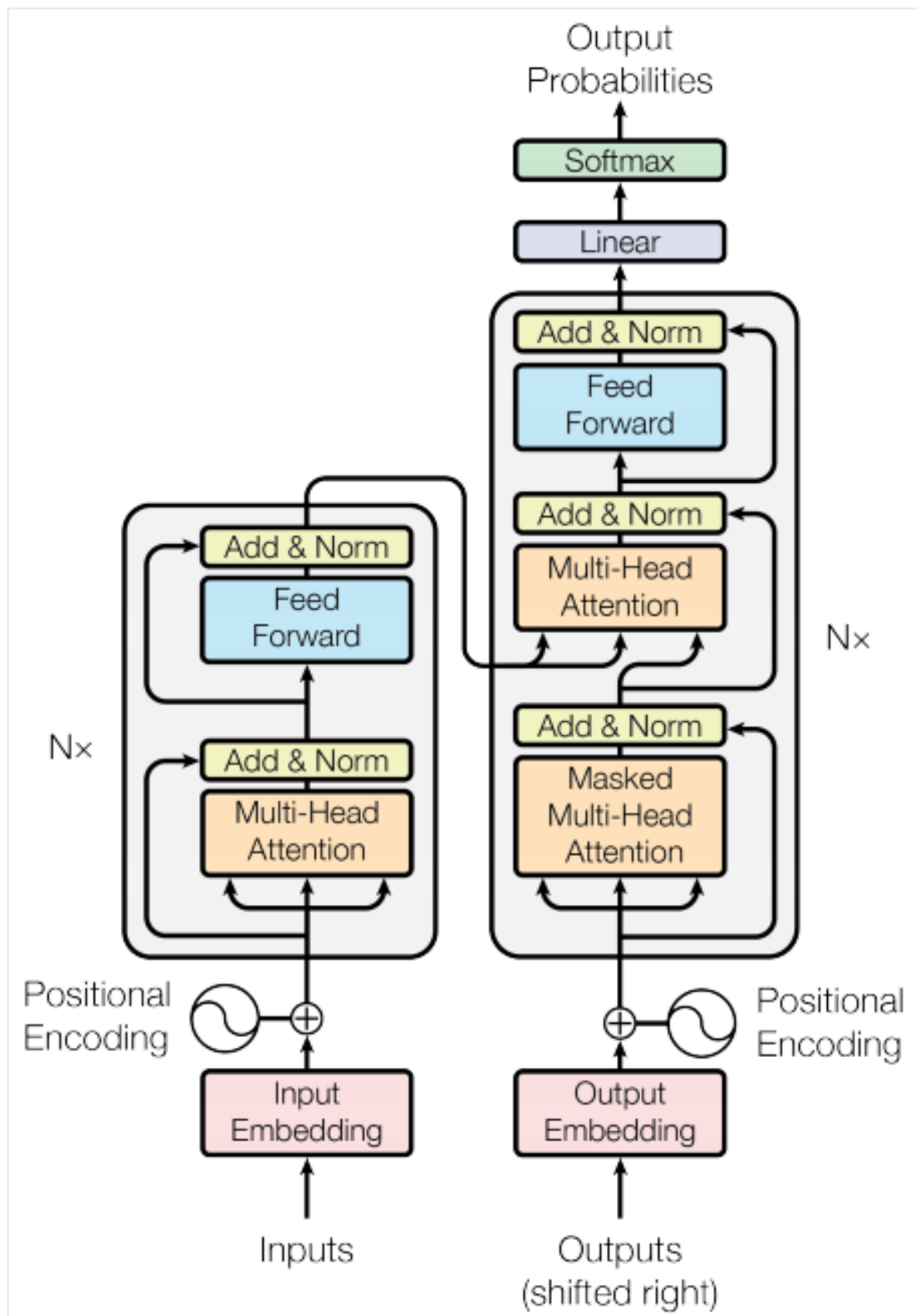
Best for:	Understanding	Generation	Transformation
-----------	---------------	------------	----------------

Input:	full sequence	partial sequence	full sequence
--------	---------------	------------------	---------------

Output:	vectors	next token	full sequence
---------	---------	------------	---------------

Examples:	BERT	GPT, Claude	T5, BART
	RoBERTa	LLaMA	Original (2017)

Used in	Classification	Chatbots	Translation
practice for:	Sentiment	Code gen	Summarization
	NER	Creative text	Q&A



Reinforcement Learning from Human Feedback

Pretraining: MASSIVE data, no human labels $\rightarrow$ learns language

SFT: SMALL data, human labels → learns to follow instructions
RLHF: SMALL data, human preferences → learns to be helpful and safe

Concrete dataset sizes used in practice

Model	Stage	Dataset size
GPT-3 pretraining	Raw text	570 GB of text (~300B tokens)
InstructGPT SFT	Conversations	~13,000 human written examples
InstructGPT RLHF	Preference pairs	~33,000 human comparisons
Claude pretraining	Raw text	Trillions of tokens (estimated)
Claude fine tuning	Constitutional AI	Proprietary

The core idea of Attention

For each token, the model asks **3 questions**:

Letter	Name	Question
Q	Query	<i>"What am I looking for?"</i>
K	Key	<i>"What do I contain / offer?"</i>
V	Value	<i>"What do I actually give if selected?"</i>

One more thing — what Anthropic does differently

Instead of pure RLHF, Anthropic uses **Constitutional AI (CAI)** for Claude:

RLHF:

human raters evaluate responses → reward model → train main model

Constitutional AI:

1. Define a "constitution" — a set of principles ("be helpful", "avoid harm", "be honest"...)
2. Model critiques its OWN responses against the constitution
3. Model revises its own responses
4. Revised responses used as training data

5. Reward model trained on AI feedback, not just human feedback

GPT (OpenAI):

Random weights



Pretraining (raw text, trillions of tokens)



SFT (human written conversations, ~13k examples)



Reward model training (human preference pairs, ~33k examples)



PPO optimization (RL against reward model + KL penalty)



ChatGPT / GPT-4

Claude (Anthropic):

Random weights



Pretraining (raw text, trillions of tokens)



SFT (human written conversations)



SL-CAI (model critiques and revises its own outputs
using constitution → new SFT data)



RLAIF (model labels preferences using constitution
→ reward model trained on AI feedback)



PPO optimization (RL against reward model + KL penalty)



Claude

The Reason and Act (ReAct) framework solves this by interleaving these two capabilities.